Experiment:2

Aim : Implement Merge Sort, Quick Sort and Binary Search

1. Merge Sort

Definition:

Merge Sort is a **divide and conquer** sorting algorithm that divides the array into halves, recursively sorts them, and then merges the sorted halves. It is efficient and works well on large datasets.

Algorithm:

1. If the array has 1 or 0 elements, it is already sorted.
2. Divide the array into two halves.
3. Recursively apply Merge Sort to both halves.
4. Merge the two sorted halves into a single sorted array.

Program:

```
def merge_sort(arr):

    if len(arr) > 1:

        mid = len(arr) // 2

        left = arr[:mid]

        right = arr[mid:]

        merge_sort(left)

        merge_sort(right)

        i = j = k = 0

        # Merge the two halves

        while i < len(left) and j < len(right):

            if left[i] < right[j]:

                arr[k] = left[i]

                i += 1

            else:

                arr[k] = right[j]
```

Dept of AIML

```
            j += 1
        k += 1
    # Copy remaining elements
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
    return arr
# Example
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

OUTPUT:

Sorted array: [3, 9, 10, 27, 38, 43, 82]

2. Quick Sort

Definition:
Quick Sort is a divide and conquer algorithm that selects a "pivot" element, partitions the array such that elements less than the pivot come before it and elements greater come after, and recursively sorts the partitions.

Algorithm:

1. Choose a pivot element (commonly the last element).
2. Partition the array into two subarrays: elements smaller than pivot and elements greater than pivot.
3. Recursively apply Quick Sort to both subarrays.
4. Combine the sorted subarrays.

program :

Dept of AIML

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[-1]
        left = [x for x in arr[:-1] if x <= pivot]
        right = [x for x in arr[:-1] if x > pivot]
        return quick_sort(left) + [pivot] + quick_sort(right)
# Example
arr = [10, 7, 8, 9, 1, 5]
sorted_arr = quick_sort(arr)
print("Sorted array:", sorted_arr)
```

OUTPUT:

Sorted array: [1, 5, 7, 8, 9, 10]

3. Binary Search

Definition:
Binary Search is an efficient search algorithm for finding the position of a target value in a sorted array. It repeatedly divides the search interval in half.

Algorithm:

1. Initialize `low = 0` and `high = n-1`.
2. Find the middle index `mid = (low + high) // 2`.
3. If `arr[mid]` equals the target, return `mid`.
4. If `arr[mid]` > target, search the left subarray (`high = mid - 1`).
5. If `arr[mid]` < target, search the right subarray (`low = mid + 1`).
6. Repeat steps 2–5 until the target is found or the array is exhausted.

Program:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1  # Not found

# Example
arr = [1, 3, 5, 7, 9, 11, 13]
target = 7
result = binary_search(arr, target)
print("Element found at index:", result)
```

OUTPUT:

Element found at index: 3

Dept of AIML